

VERSION 1.0
17 JANUARI 2025



[PENGGALIAN DATA]

MODUL 1 - DATA STRUCTURE IN PYTHON

DISUSUN OLEH:
ALVIYA LAELA LESTARI
KIARA AZZAHRA

DIAUDIT OLEH:
YUFIS AZHAR, S.KOM, M.KOM

LAB. INFORMATIKA
UNIVERSITAS MUHAMMADIYAH MALANG

[PENGALIAN DATA]

PERSIAPAN MATERI

Praktikan mempelajari konsep dasar bahasa pemrograman Python.

TUJUAN

1. Mahasiswa mampu memahami dasar bahasa pemrograman python
2. Mahasiswa mampu memahami tipe-tipe struktur data pada bahasa pemrograman python
3. Mahasiswa mampu memahami list, array, tuple dan dataframe

PERSIAPAN SOFTWARE/APLIKASI

1. Hardware
 - Laptop/Komputer
2. Software (Pilih salah satu)
 - Visual Studio Code, Google Collaboratory, PyCharm, Jupyter Notebook
3. Tutorial
 - Menggunakan Google Colab :
https://youtu.be/tpDjhSzJor4?si=bFjSP_mf3oB5x520
 - Menggunakan Jupyter di VS Code :
<https://youtu.be/HOg13z1wl1A?si=RhEY2QY5bir-dzEH>
 - Menggunakan Jupyter Notebook :
https://youtu.be/HQaCF2F_MR0?si=k6ubuGuCg-VS0tQe
 - Instal Anaconda : https://youtu.be/XyudQ3B3OWI?si=7qugN_vTxAM30qDt
4. Source Code:
<https://colab.research.google.com/drive/13EtSN6B7e4y1jTC9jUwKKp3D81N9UC9y?usp=sharing>

1. List dan Tuple

Pada semester sebelumnya, kalian sudah mempelajari terkait Pemrograman Fungsional yang membahas tipe data yang ada dalam Python, termasuk sequence type. Sequence adalah kumpulan nilai yang diakses melalui indeks dan memiliki beberapa tipe data utama, seperti array. Namun, di Python, tipe data sequence sedikit berbeda dari array pada umumnya. Data sequence lebih dinamis dibandingkan array, kita bisa menambahkan elemen sebanyak yang kita inginkan tanpa batasan ukuran seperti pada array. Selain itu, sequence juga dapat menyimpan elemen dengan tipe data yang berbeda.

Pada modul ini, kita akan membahas ulang dua tipe data sequence yang sangat sering digunakan, yaitu List dan Tuple. Keduanya memiliki fitur yang mirip namun dengan karakteristik tertentu yang membedakannya.

1.1 List

List dalam Python adalah salah satu tipe data sequence yang paling fleksibel dan sering digunakan. List ini berisi kumpulan elemen yang berurutan (ter-index) dan dapat diubah (mutable). Berikut adalah beberapa poin penting tentang list dalam Python:

a. Mutable (Dapat Diubah)

Kita tetap bisa melakukan perubahan pada data dalam list karena sifat dari list yang dapat diubah/mutable.

b. Ordered (Berurutan)

Maksud dari berurutan disini adalah bahwa untuk mengakses data dalam list akan dimulai berurutan sesuai dengan urutan indexnya.

```
list = ['a', 'b', 'c', 'd', 'e']
print(list[0])
print(list[1])
print(list[2])
```

Output:

```
a
b
c
```

c. Heterogeneous (Banyak Tipe Data)

List tidak kaku yang berarti List mampu menyimpan value dengan bermacam-macam tipe data seperti contoh di bawah ini:

```
list = ["data", 7, 1.25, 8, "mining", 5, False]
print(list)

# Output: ['data', 7, 1.25, 8, 'mining', 5, False]
```

d. Allow Duplicates (Duplikasi)

List memperbolehkan data yang memiliki value sama karena List dipengaruhi oleh index.

```
list = [1, "adi", True, 1, "adi", True]
print(list)

# Output: [1, "adi", True, 1, "adi", True]
```

e. Dinamis

List memperbolehkan operasi pada data. Operasi yang dapat digunakan pada list:

- Menambahkan item

extend(): menambahkan semua elemen iterable (seperti list, tuple atau string) satu per satu di posisi terakhir list.

```
list = [1, "adi", True]
list.extend(["budi", 7, 2.15])
print(list)

# Output : [1, "adi", True, "budi", 7, 2.15]
```

append(): menambahkan satu elemen yang diperlakukan sebagai satu objek tunggal di posisi terakhir list.

```
list = [1, "adi", True]
list.append(["budi", 7, 2.15])
print(list)

# Output : [1, "adi", True, ["budi", 7, 2.15]]
```

insert(): menambahkan list baru ke posisi yang ditentukan.

Dalam kode di bawah, kita akan menyisipkan list baru sebelum indeks ke-2, yaitu sebelum elemen True.

```
list = [1, "adi", True]
list.insert(2, ["budi", 7, 2.15])
print(list)

# Output : [1, 'adi', ['budi', 7, 2.15], True]
```

- Menghapus item

remove(item): menghapus item pertama dalam list yang cocok dengan nilai yang diberikan.

```
fruits = ["apple", "banana", "cherry", "apple"]
fruits.remove("apple")
print(fruits)

# Output : ['banana', 'cherry', 'apple']
```

Jika perintah remove kembali dijalankan, maka item 'apple' selanjutnya akan terhapus.

```
fruits.remove("apple")
print(fruits)

# Output : ['banana', 'cherry']
```

pop(index): menghapus item berdasarkan index dan mengembalikan item yang dihapus. Jika nilai index tidak diberikan, maka item terakhir yang akan dihapus.

```

numbers = [10, 20, 30, 40, 50, 60, 70]

tanpa_index = numbers.pop()
print("Pop tanpa index:", numbers)
# Output: Pop tanpa index: [10, 20, 30, 40, 50, 60]

print("Removed:", tanpa_index)
# Output: Removed: 70

index = numbers.pop(2)
print("Pop dengan index:", numbers)
# Output: Pop dengan index: [10, 20, 40, 50, 60]

print("Removed:", index)
# Output: Removed: 30

```

clear(): menghapus seluruh item dari list, sehingga list menjadi kosong.

```

names = ["Alice", "Bob", "Charlie"]
names.clear()
print(names)

# Output: []

```

- Menemukan item

index(item): mencari index dari item pertama yang cocok dengan nilai yang diberikan.

```

colors = ["red", "blue", "green", "blue"]
index = colors.index("blue")
print(index)

# Output: 1

```

- Menghitung item

count(item): menghitung jumlah kemunculan item dalam list.

```
scores = [10, 20, 10, 30, 10, 20, 10, 70, 10]
count = scores.count(10)
print(count)

# Output: 5
```

- Mengurutkan item

sort(): mengurutkan item dalam list secara ascending secara default, atau descending jika menggunakan parameter reverse=True.

note: untuk mengurutkan item hanya bisa satu tipe data.

```
numbers = [10, 20, 10, 30, 10, 20, 10, 70, 10]
numbers.sort() # Ascending
print(numbers)
# Output: [10, 10, 10, 10, 10, 20, 20, 30, 70]

numbers.sort(reverse=True) # Descending
print(numbers)
# Output: [70, 30, 20, 20, 10, 10, 10, 10, 10]
```

- Membalik urutan

reverse(): membalikkan urutan elemen dalam list tanpa memperhatikan nilai elemen.

```
letters = ["a", "b", "c", "d", 1, 4, True, 2, 3, 1]
letters.reverse()
print(letters)

# Output: [1, 3, 2, True, 4, 1, 'd', 'c', 'b', 'a']
```

1.2 Tuple

Tuple adalah urutan objek dalam Python yang bersifat tidak dapat diubah (immutable). Tuple serupa dengan list, namun perbedaan utamanya adalah elemen dalam tuple tidak dapat diubah setelah didefinisikan. Tuple menggunakan tanda kurung '()', sedangkan list menggunakan tanda kurung siku '[]'. Membuat

tuple sangat mudah; kita hanya perlu menuliskan nilai-nilai yang dipisahkan oleh koma. Berikut adalah beberapa poin penting tentang tuple dalam Python:

a. Immutable(tidak dapat diubah)

Saat berhasil dibuat, tuple tidak dapat diubah, termasuk menambahkan atau menghapus elemen di dalamnya.

```
tuple = ("a", "b", "c", "d", "e")
tuple[0] = "z"
print(tuple)

# Output: TypeError: 'tuple' object does not support item assignment
```

b. Heterogeneous (Banyak Tipe Data)

Tuple mengizinkan untuk memiliki data yang berbeda tipe seperti List.

```
tuple = ("data", 7, 1.25, 8, "mining", 5, False)
print(tuple)

# Output: ('data', 7, 1.25, 8, 'mining', 5, False)
```

c. Ordered (Berurutan)

Tuple bersifat ordered seperti List yaitu untuk mengaksesnya kita bisa menggunakan index masing-masing data.

```
tuple = (1, 2, 3, 4, 5)
print(tuple[1])
print(tuple[2])
print(tuple[3])
```

```
Output:
2
3
4
```

d. Indexable

Setiap item dalam tuple dapat diakses menggunakan indeksnya.

```
t = (10, 20, 30, 40)
print(t[0])
print(t[3])
```

```
Output:
10
40
```

e. Allow Duplicates (Duplikasi)

Tuple juga memperbolehkan data yang memiliki value sama seperti List yang dipengaruhi oleh index.

```
tuple = (True, 7, "budi", "budi", 7, True)
print(tuple)

# Output: (True, 7, "budi", "budi", 7, True)
```

2. NumPy dan Array

NumPy (Numerical Python) adalah library Python yang digunakan untuk mengelola array berukuran besar, termasuk array multidimensi. Library ini juga menyediakan berbagai fungsi matematika untuk mempermudah perhitungan kompleks pada array. Array di NumPy adalah struktur data berbentuk grid yang dirancang khusus untuk menyimpan data numerik. Dibandingkan dengan list Python, array NumPy lebih cepat dan efisien dalam melakukan operasi perhitungan karena telah dioptimalkan.

2.1 Array NumPy vs List Python

Array NumPy	List Python
Ukuran Tetap	Ukuran Dinamis
Elemen harus memiliki tipe data yang sama	Tipe elemen bisa beragam
Operasi matematika element-wise lebih	Tidak mendukung operasi langsung

cepat	
-------	--

2.2 Membuat Array dengan NumPy

a. Array 1D

Membuat array satu dimensi yang digunakan untuk menyimpan daftar data linear.

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5])
print(arr)
```

```
# Output: [1 2 3 4 5]
```

b. Array 2D

Membuat array dua dimensi yang menyerupai matriks atau tabel data dengan baris dan kolom.

```
matrix = np.array([[1, 2], [3, 4], [5, 6]])
print(matrix)
```

```
# Output:
# [[1 2]
#  [3 4]
#  [5 6]]
```

c. Array 3D

Membuat array tiga dimensi yang bisa menyimpan data kompleks seperti tensor dalam pembelajaran mesin.

```
array_3d = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])
print(array_3d)
# Output:
# [[[1 2]
#      [3 4]]
#  [[5 6]
#      [7 8]]]
```

2.3 Properti Array

a. Dimensi Array (ndim)

Memberikan informasi tentang jumlah dimensi array, misalnya 1D, 2D, atau 3D.

b. Bentuk Array (shape)

Menunjukkan jumlah elemen pada setiap dimensi (misalnya, array 2×3 memiliki shape (2, 3)).

c. Jumlah Elemen (size)

Mengembalikan total elemen dalam array.

d. Tipe Data (dtype)

Menjelaskan tipe data yang digunakan oleh array, seperti int32 atau float64.

```
array = np.array([1, 2, 3], [4, 5, 6])

print("Dimensi:", array.ndim)           # Output: Dimensi: 2
print("Shape:", array.shape)            # Output: Shape: (2, 3)
print("Jumlah elemen:", array.size)     # Output: Jumlah elemen: 6
print("Tipe data:", array.dtype)        # Output: Tipe data: int64
```

2.4 Operasi Matematika pada Array

a. Operasi Element-wise

Melakukan operasi matematika seperti penjumlahan, perkalian, atau pemangkatan pada elemen array secara langsung, elemen demi elemen.

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

print(a + b)    # Output: [5 7 9]
print(a * b)    # Output: [4 10 18]
print(a ** 2)   # Output: [1 4 9]
```

b. Operasi Matriks

Melakukan operasi aljabar linear yang berguna untuk manipulasi data numerik skala besar.

- Penjumlahan dan Pengurangan pada Matriks

Penjumlahan dan pengurangan dua matriks hanya bisa dilakukan jika keduanya memiliki ukuran yang sama.

```
import numpy as np

mat1 = np.array([[1, 2], [3, 4]])
mat2 = np.array([[5, 6], [7, 8]])

# Penjumlahan
print(mat1 + mat2)
# [[ 6  8]
#  [10 12]]

# Pengurangan
print(mat2 - mat1)
# [[ 4  4]
#  [ 4  4]]
```

- Perkalian pada Matriks

Perkalian matriks dapat dilakukan sebagai:

Perkalian Element-wise: Mengalikan elemen yang bersesuaian di dua matriks (operator `*` di NumPy).

Perkalian Matriks: Perkalian linear standar di mana elemen dihitung sebagai hasil perkalian baris dan kolom (menggunakan fungsi `np.dot()` atau operator `@`).

```
import numpy as np

mat1 = np.array([[1, 2], [3, 4]])
mat2 = np.array([[5, 6], [7, 8]])

# Perkalian element-wise
print(mat1 * mat2)
# [[ 5 12]
#  [21 32]]

# Perkalian matriks
print(np.dot(mat1, mat2))
# [[19 22]
#  [43 50]]
```

2.5 Indexing dan Slicing

Seperti list Python, NumPy array mendukung akses elemen individu dan subset elemen menggunakan indeks. Fitur ini sangat fleksibel, bahkan memungkinkan manipulasi sebagian data tanpa mempengaruhi data aslinya.

```
array = np.array([1, 2, 3, 4, 5])

# Mengakses elemen
print(array[0]) # 1
print(array[-1]) # 5

# Slicing
print(array[1:4]) # [2, 3, 4]
print(array[:3]) # [1, 2, 3]
```

2.6 Mengubah Bentuk (Reshape)

Reshaping digunakan untuk mengubah bentuk array tanpa mengubah datanya. Misalnya, Anda dapat mengubah array 1D menjadi 2D atau sebaliknya.

Fungsi Utama:

- `reshape()`: Mengubah bentuk array menjadi dimensi yang diinginkan.
- `flatten()`: Mengubah array multi-dimensi menjadi 1D.

```
import numpy as np

# Array 1D
arr = np.array([1, 2, 3, 4, 5, 6])

# Mengubah menjadi array 2D (2 baris, 3 kolom)
reshaped_arr = arr.reshape(2, 3)
print("Reshaped Array:")
print(reshaped_arr)
# Output:
# [[1 2 3]
#  [4 5 6]]

# Mengubah kembali ke array 1D
flattened_arr = reshaped_arr.flatten()
print("\nFlattened Array:")
print(flattened_arr)
# Output: [1 2 3 4 5 6]
```

2.7 Menggabungkan Array (Concatenation)

Penggabungan array berarti menggabungkan beberapa array menjadi satu, baik secara horizontal (dalam baris) maupun vertikal (dalam kolom).

Fungsi Utama:

- `np.concatenate()`: Menggabungkan array pada axis tertentu.
- `np.vstack()`: Menggabungkan secara vertikal (baris bertambah).
- `np.hstack()`: Menggabungkan secara horizontal (kolom bertambah)

```
arr1 = np.array([[1, 2], [3, 4]])
arr2 = np.array([[5, 6], [7, 8]])

# Menggabungkan secara vertikal
vertical_concat = np.vstack((arr1, arr2))
print("Vertical Concatenation:")
print(vertical_concat)
# Output:
# [[1 2]
#  [3 4]
#  [5 6]
#  [7 8]]

# Menggabungkan secara horizontal
horizontal_concat = np.hstack((arr1, arr2))
print("\nHorizontal Concatenation:")
print(horizontal_concat)
# Output:
# [[1 2 5 6]
#  [3 4 7 8]]
```

2.8 Pemisahan (Splitting)

Pemisahan array berarti membagi satu array menjadi beberapa array kecil dengan jumlah elemen tertentu.

Fungsi Utama:

- `np.split()`: Membagi array pada index tertentu.
- `np.vsplit()`: Membagi secara vertikal.
- `np.hsplit()`: Membagi secara horizontal.

```
# Membagi array 2D menjadi dua
arr = np.array([[1, 2, 3, 4], [5, 6, 7, 8]])

# Membagi secara horizontal menjadi dua bagian
horizontal_split = np.hsplit(arr, 2)
print("Horizontal Splitting:")
print(horizontal_split[0]) # Bagian pertama
print(horizontal_split[1]) # Bagian kedua
# Output:
# [[1 2]
#  [5 6]]
# [[3 4]
#  [7 8]]

# Membagi secara vertikal menjadi dua bagian
vertical_split = np.vsplit(arr, 2)
print("\nVertical Splitting:")
print(vertical_split[0]) # Bagian pertama
print(vertical_split[1]) # Bagian kedua
# Output:
# [[1 2 3 4]]
# [[5 6 7 8]]
```

3. Pandas Dataframe

Pandas adalah library Python yang sangat berguna untuk analisis dan pengolahan data, cocok untuk menangani dataset dari ukuran menengah hingga besar. Dalam Pandas, terdapat dua jenis struktur data utama:

- Series:** Array satu dimensi yang dapat menyimpan berbagai jenis data dan memiliki label axis.
- DataFrame:** Struktur data dua dimensi, mirip dengan tabel dengan baris dan kolom, yang dapat menyimpan berbagai jenis data dan juga memiliki label axis.

Beberapa keunggulan dan fitur utama dari Pandas adalah:

- Pemuatan Data yang Cepat:** Dapat dengan mudah memuat data dari berbagai sumber seperti file CSV, Excel, atau database.
- Manipulasi Data yang Efisien:** Menawarkan berbagai fungsi dan metode untuk membersihkan, menyortir, dan menyusun ulang data dengan cara yang sangat efektif.
- Penggabungan Data:** Memungkinkan penggabungan dan pengelompokan data dengan mudah, serupa dengan SQL join operations.
- Berbagai Fungsi Bawaan:** Menyediakan banyak fungsi untuk mengagregasi, transformasi, dan analisis data yang komprehensif.

3.1 Membuat Dataframe dari Array Sederhana

Membuat Dataframe dari Array Sederhana adalah salah satu cara dasar dalam menggunakan library Pandas untuk memanipulasi dan menganalisis data.

```
import pandas as pd

dataframe = pd.DataFrame()
print(dataframe) # Dataframe kosong

words = ['ini', 'adalah', 'dataframe']

dataframe = pd.DataFrame(words)
print("\n", dataframe)

# Output:
# Empty DataFrame
# Columns: []
# Index: []

#           0
# 0         ini
# 1      adalah
# 2    dataframe
```

3.2 Membuat Dataframe dari Dictionary Pandas

Dictionary di Python adalah struktur data yang terdiri dari pasangan key dan value, di mana key dapat digunakan sebagai nama kolom di dalam DataFrame. Dengan menggunakan dictionary, pengguna dapat dengan mudah

mengorganisasi data dan membuat Dataframe yang rapi, memungkinkan manipulasi dan analisis data yang lebih efisien.

```
import pandas as pd

myvar = pd.DataFrame()
print(myvar) # Dataframe kosong

mydataset = {
    'Nama': ["Andi", "Budi", "Cinta", "Doni", "Elsa"],
    'Tinggi': [168, 170, 150, 154, 188]
}

# Memanggil dataframe constructor pada list
myvar = pd.DataFrame(mydataset)
print(myvar)

# Output:
# Empty DataFrame
# Columns: []
# Index: []

# Nama  Tinggi
# 0    Andi    168
# 1    Budi    170
# 2   Cinta    150
# 3    Doni    154
# 4    Elsa    188
```

CODELAB

Codelab dikerjakan di Google Collab masing-masing!

CODELAB 1

- Buatlah 12 data tuple yang berisi masing-masing 4 data int, float, dan string dengan urutan acak.
- Ubahlah data tuple tersebut menjadi list.
- Tambahkan 2 data baru di index sesuai dengan NIM akhir kalian (jika NIM akhir adalah 0, maka index ke-10).
- Hapus tipe data float pada list.
- Urutkan elemen pada list secara descending.

CODELAB 2

- Buatlah array 2D berdasarkan 6 digit terakhir NIM kalian.
- Lakukan operasi penambahan, pengurangan dan perkalian pada matriks yang kalian buat.
- Ubahlah matriks tersebut menjadi bentuk 1D.
- Lakukan slicing pada matriks.

CODELAB 3

- Buatlah sebuah series yang berisi dictionary yang memuat nama produk dan ID produk.
- Buatlah sebuah dataframe yang berisi dictionary yang memuat 4 kolom dan setiap kolom berisi 5 data. **Tidak boleh ada kesamaan antar praktikan.**
- Tambahkan 1 kolom baru.
- Urutkan dataframe, misalnya berdasarkan abjad jika kolom berupa Nama.

TUGAS

TUGAS 1

Buatlah sebuah fungsi yang menerima list sebagai parameter dan melakukan operasi berikut:

- Tambahkan 10 elemen numerik (int dan float) secara acak.
- Hapus elemen di indeks sesuai NIM (ganjil/genap).
- Balik urutan elemen dalam list.
- Hapus elemen dengan nilai terbesar dari list tersebut.
- Urutkan elemen pada list secara descending.

TUGAS 2

Buatlah sebuah tuple yang berisi dictionary dengan 10 pasangan key value mata kuliah dan nilai. Dari 10 pasangan key value tersebut, buatlah minimal 3 diantaranya memiliki nilai kosong. Lakukan operasi berikut:

- Hapus entri yang tidak lengkap atau memiliki nilai kosong.
- Hitung jumlah entri yang tersisa setelah penghapusan.

3. Urutkan pasangan key value tersebut berdasarkan nilai secara ascending.
4. Carilah mata kuliah dengan nilai tertinggi.
5. Hitunglah rata-rata semua nilai mata kuliah.

TUGAS 3

[NIM GENAP]

Buatlah dataframe yang terdiri dari 10 baris dan 6 kolom, dengan informasi sebagai berikut:

	Brand	Cabang	Harga (ribu)	Stock	Status
1	Avoskin	Surabaya	150	175	Sold
2	G2G	Jakarta Pusat	75	160	Available
3	Skintific	Bandung	120	120	Available
4	Wardah	Bandung	85	300	Sold
5	Emina	Jakarta	75	230	Available
6	Somethinc	Bandung	120	200	Available
7	Azarine	Surabaya	50	180	Available
8	Lacoco	Bandung	90	100	Sold
9	Y.O.U	Jakarta Pusat	55	180	Available
10	Madame Gie	Surabaya	50	200	Sold

Lakukan analisis berikut pada DataFrame tersebut:

1. Tampilkan ringkasan statistik deskriptif yang mencakup nilai minimum, maksimum, jumlah, rata-rata, standar deviasi, serta nilai kuartil 1, 2, dan 3.
2. Tentukan 3 brand dengan penghasilan terbesar (harga x stock).
3. Kelompokkan brand dengan harga yang sama.

4. Buatlah kolom baru yang berisi kategori 'Murah', 'Sedang', dan 'Mahal'. Kategori ini didasarkan pada aturan:
- Harga di bawah kuartil 1 dikategorikan sebagai 'Murah'
 - Harga di atas kuartil 3 dikategorikan sebagai 'Mahal'
 - Sisanya dikategorikan sebagai 'Sedang'.

[NIM GANJIL]

Buatlah dataframe yang terdiri dari 10 baris dan 6 kolom, dengan informasi sebagai berikut:

	Brand	Tahun Rilis	Terjual	Harga (Juta)	Jenis Penjualan
1	Apple	2015	150	19	Online
2	Xiaomi	2017	75	9	Offline
3	Samsung	2018	120	10	Online
4	Infinix	2015	85	9	Online
5	Oppo	2017	75	4	Offline
6	Asus	2020	120	9	Online
7	Lenovo	2019	50	15	Offline
8	Huawei	2019	90	15	Online
9	Google Pixel	2020	55	15	Offline
10	Sony	2017	50	11	Offline

Lakukan analisis berikut pada DataFrame tersebut:

1. Tampilkan ringkasan statistik deskriptif yang mencakup nilai minimum, maksimum, jumlah, rata-rata, standar deviasi, serta nilai kuartil 1, 2, dan 3.
2. Tentukan 3 brand dengan penghasilan terbesar (harga x terjual).
3. Kelompokkan brand dengan jenis penjualan yang sama.

4. Buatlah kolom baru yang berisi kategori 'Murah', 'Sedang', dan 'Mahal'. Kategori ini didasarkan pada aturan:
- Harga di bawah kuartil 1 dikategorikan sebagai 'Murah'
 - Harga di atas kuartil 3 dikategorikan sebagai 'Mahal'
 - Sisanya dikategorikan sebagai 'Sedang'.

PERINGATAN: Terdapat Kemiripan Source Code Yang Signifikan Dengan Praktikan Lain, Nilai Maksimal D

KRITERIA & DETAIL PENILAIAN

Detail		Bobot Nilai
Codelab	Codelab 1	5
	Codelab 2	5
	Codelab 3	5
Tugas	Tugas 1	15
	Tugas 2	15
	Tugas 3	15
Pemahaman Materi & Ketepatan Jawaban		40